МИКРОСЕРВИСНАЯ АРХИТЕКТУРА

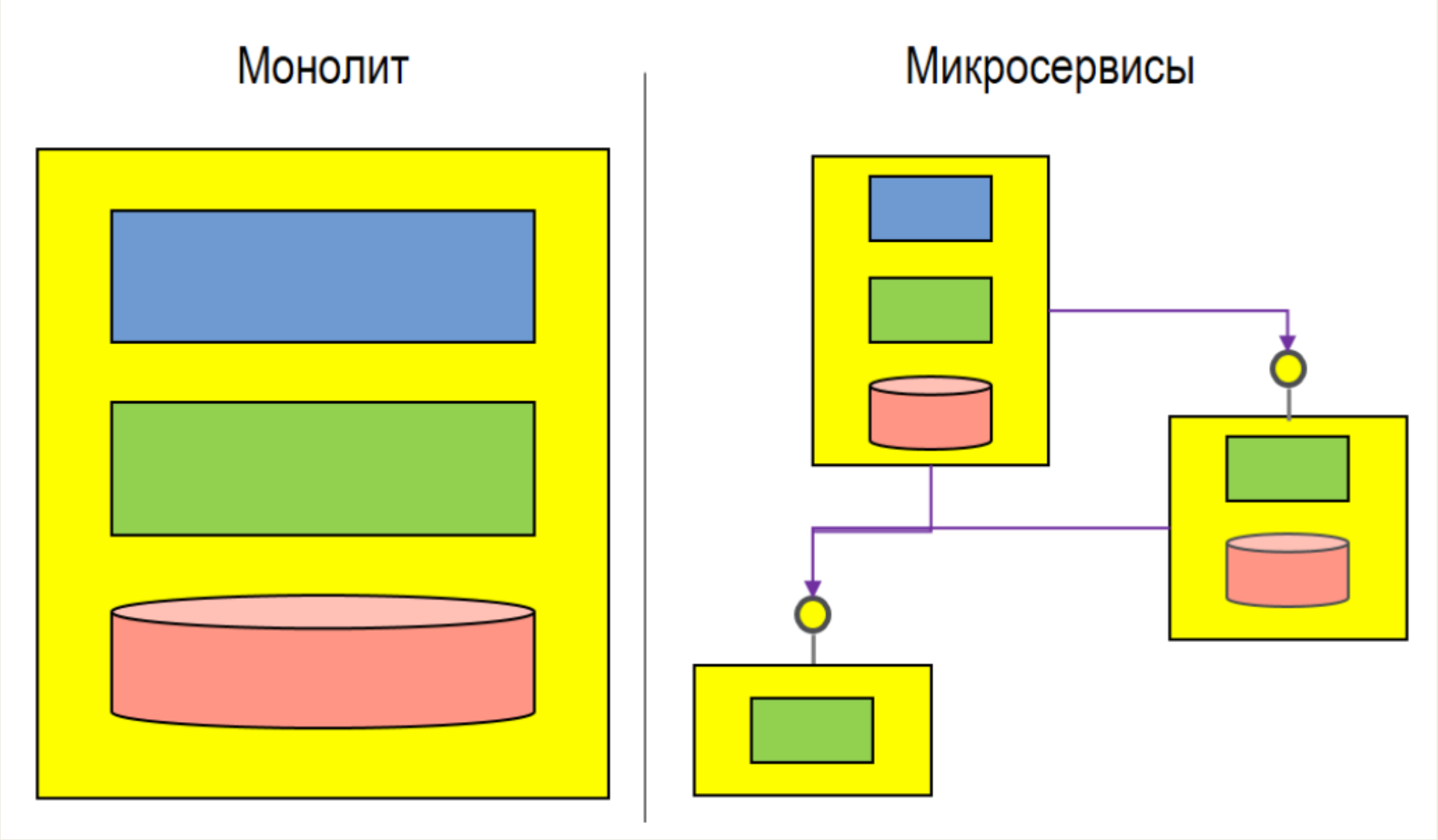
Максимально распределенная система

Микросервисная архитектура (MSA)

- MSA — принципиальная организация распределенной системы на основе микросервисов и их взаимодействия друг с другом и со средой по сети, а также принципов, направляющих проектирование архитектуры, её создание и эволюцию.

https://habr.com/ru/companies/raiffeisenbank/articles/346380/

MSA vs Monolit



Основные свойства микросервиса

1. Он небольшой.
2. Он независимый.
3. Он строится вокруг бизнес-потребности и использует ограниченный контекст *(Bounded Context)*.
4. Он взаимодействует с другими микросервисами по сети на основе паттерна *Smart endpoints and dumb pipes.*
5. Его распределенная суть обязывает использовать подход *Design for failure*.
6. Централизация ограничена сверху на минимуме.
7. Процессы его разработки и поддержки требуют автоматизации.
8. Его развитие итерационное.

Небольшой

1. Один сервис может развивать одна команда не более чем из дюжины человек.
2. Команда из полудюжины человек может развивать полдюжины сервисов.
3. Контекст (не только бизнеса, но и разработки) одного сервиса помещается в голове одного человека.
4. Один сервис может быть полностью переписан одной командой за одну Agile-итерацию.

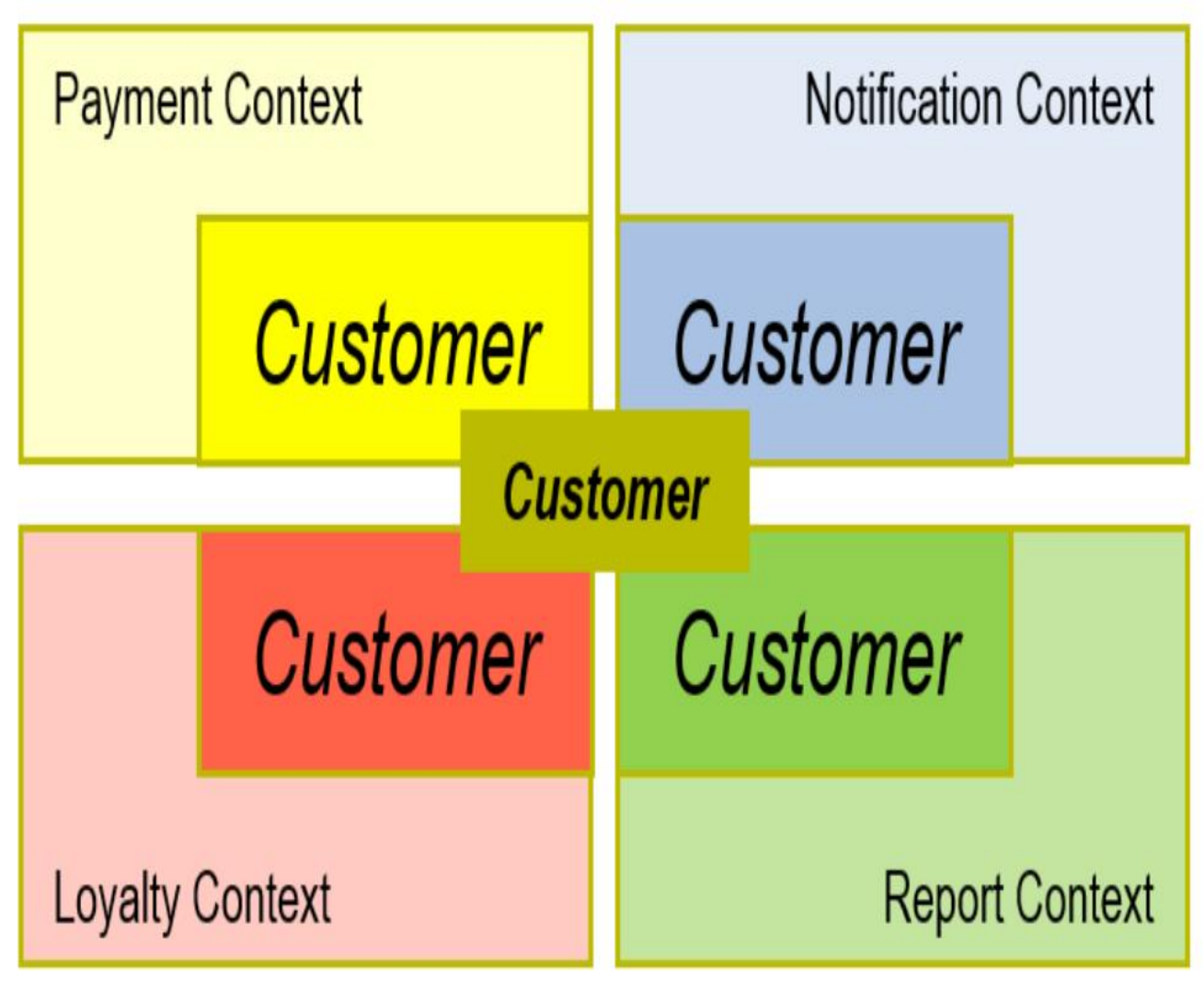
Независимый

Микросервисная архитектура — воплощение паттернов High Cohesion (связанность бизнес-логики) и Low Coupling (**минимальная связанность**). То **есть** система должна состоять и слабо связанных классов, которые содержат связанную бизнес — логику
Всё, что противоречит этому, отвергается беспощадно.
В противном случае команду ждут большие проблемы.
Так что микросервис обязан быть независимым компонентом.

Компонент — это единица ПО, код которой может быть независимо заменен или обновлен.

https://habr.com/ru/companies/otus/articles/505852/

Бизнес-потребность



Определение его границ — самый важный шаг. От этого будет зависеть вся дальнейшая жизнь микросервиса, и это серьёзно повлияет на жизнь команды, отвечающей за него.

Основной принцип определения зоны ответственности микросервиса — сформировать её вокруг некоторой бизнес-потребности. И чем она компактнее, чем формализованней её взаимоотношения с другими областями, тем проще создать новый микросервис. В общем, довольно стандартный посыл. На нем основывается создание любых других компонентов. Вопрос только в том, чтобы в дальнейшем выдержать эту зону ответственности, что мы и обсуждали в предыдущем параграфе.

Как выделять границы?

- Чтобы минимизировать ошибки при определении границ, нужно вначале их продумать. Поэтому оправданным является подход Monolith First, когда вначале систему развивают в традиционной парадигме, а когда появляются устоявшиеся области, их выделяют в микросервисы. Но всё течет и меняется. И границы тоже могут меняться. Главное, чтобы выигрыш от разбиения превышал сложности пересмотра этих границ. Такой подход к постепенному формированию набора микросервисов похож на итерационное развитие, используемое в Agile, ещё его называют «эволюционным проектированием» *(Evolutionary Design)*.

Закон Конвея

- Если организация использует монолитное приложение, то оно нарушает соответствие структуре и коммуникациям внутри организации. А команды разработчиков строятся вокруг архитектурных слоев монолита: UI, серверная логика, база данных.

  Микросервисная архитектура приводит IT и бизнес в гармонию, с точки зрения Конвея. Поскольку микросервисы формируются вокруг бизнес-потребностей конкретных бизнес-подразделений, то архитектура предприятия начинает повторять оргструктуру и каналы социальной и бизнес-коммуникации. А команды становятся кроссфункциональными и формируются вокруг этих бизнес-потребностей / бизнес-подразделений.

Пара замечаний

1. Основной сложностью разбиения монолита на микросервисы является не определение их границ. Они уже должны сформироваться и устояться. Сложность заключается в том, что локальные вызовы становятся удаленными. А это влияет не только на организацию вызовов, но и на стиль взаимодействия, так как частые вызовы уже не подходят. Скорее всего, надо пересматривать сам API, делать его более крупным, а, как следствие, пересматривать логику работы компонентов.
2. Поскольку асинхронное событийное взаимодействие — практически стандарт в микросервисной архитектуре, то надо разбираться в создании событийной архитектуры (Event Driven Architecture), а сами микросервисы должны соответствовать требованиям Reactive.

Переход с монолита в микросервисы

Шаблон «фикус-удавка», разработанный Мартином Фаулером. Рекомендуется для больших монолитов

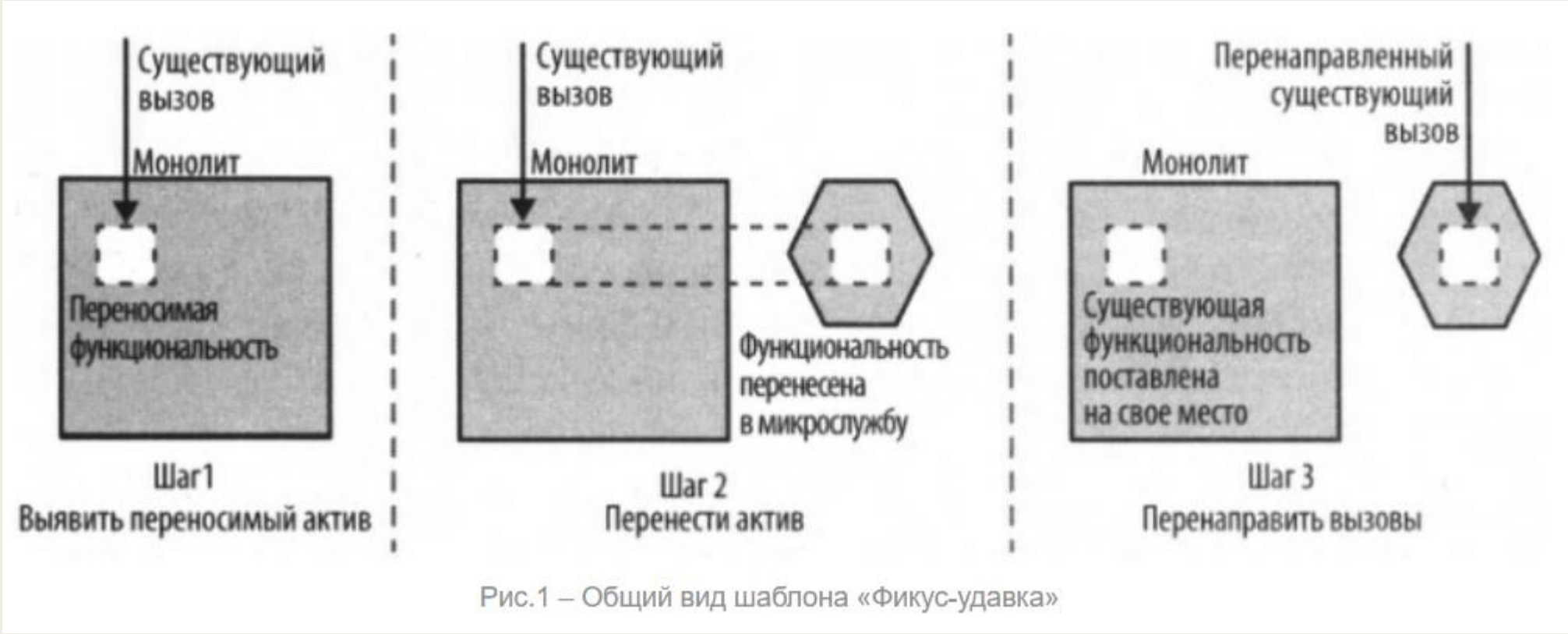
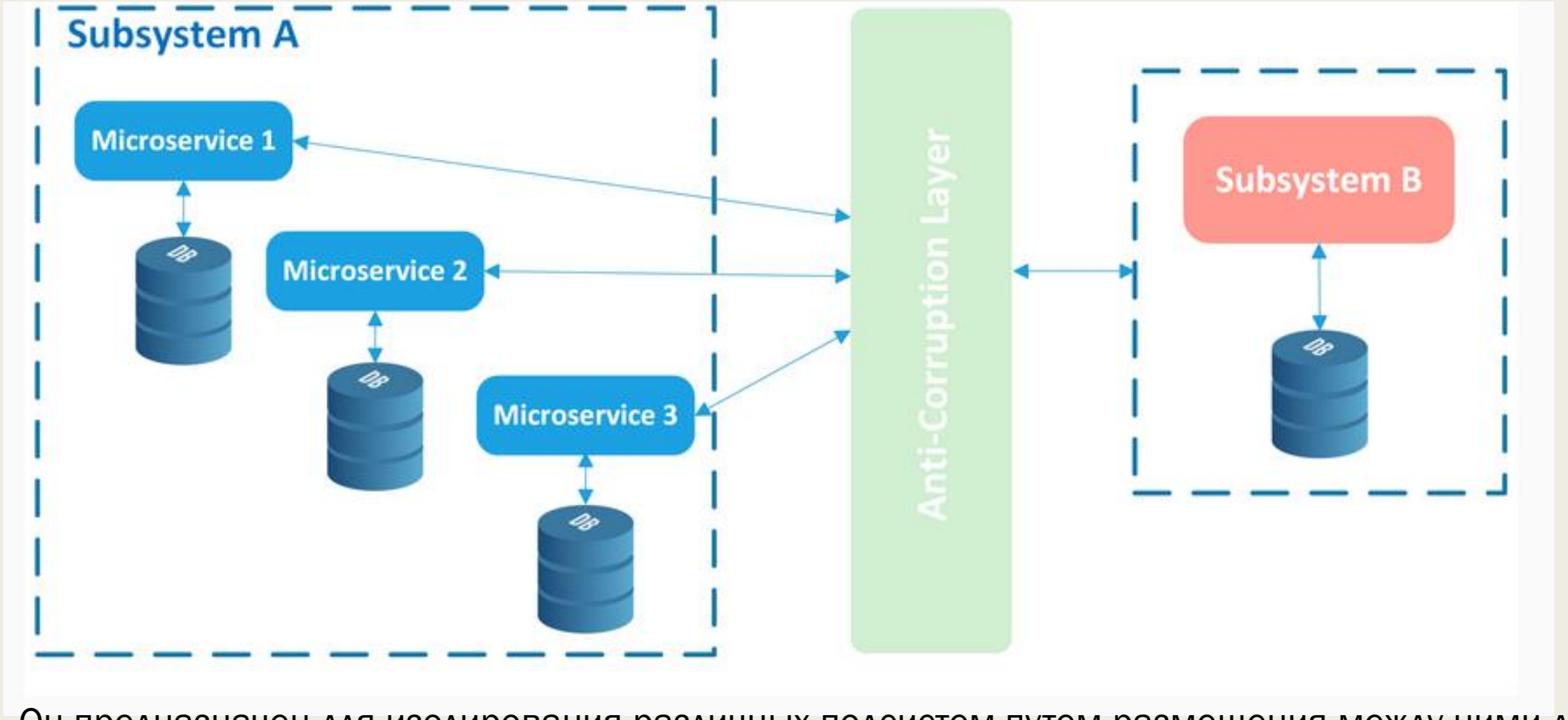


Рис.1 – Общий вид шаблона «Фикус-удавка»

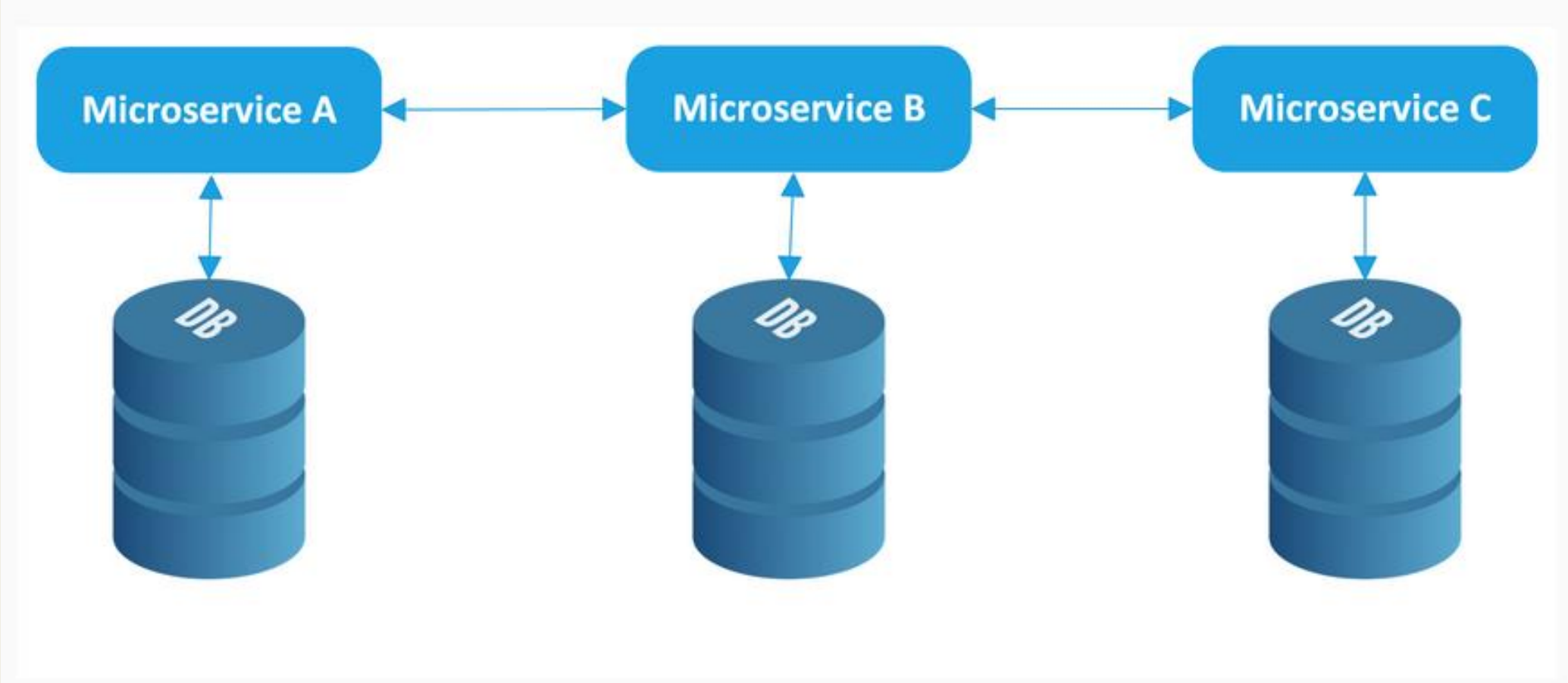
https://habr.com/ru/articles/548054/

Шаблон «Уровень защиты от повреждений» (Anti-Corruption Layer)



Он предназначен для изолирования различных подсистем путем размещения между ними дополнительного уровня, который может быть реализован как компонент приложения или независимая служба. Этот уровень связывает две подсистемы, позволяя им оставаться максимально независимыми друг от друга. Он содержит всю логику, необходимую для передачи данных в обе стороны: при взаимодействии с каждой из подсистем используется именно ее модель данных.

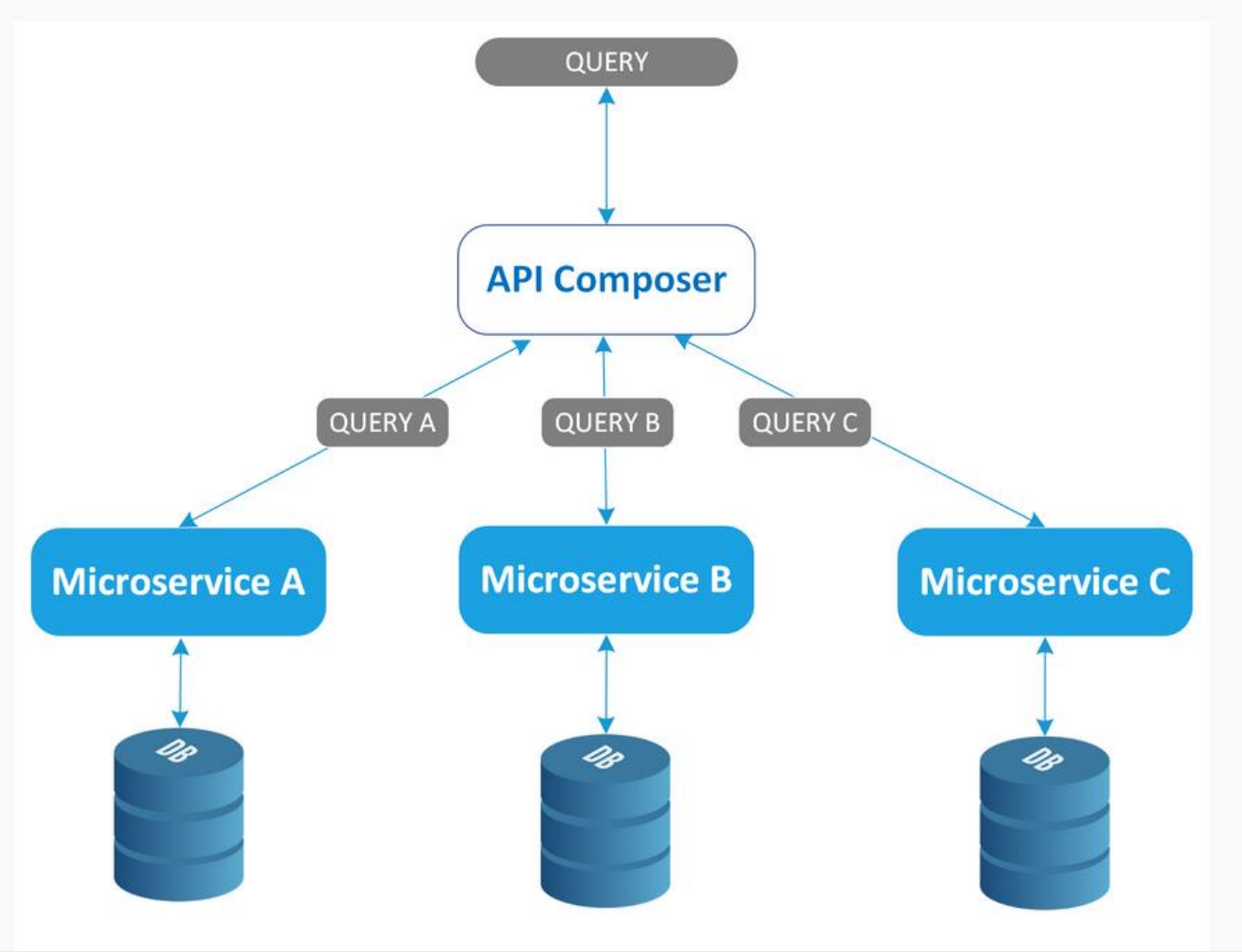
Шаблон «База данных на сервис» (Database Per Service)



Замечание

- Паттерну Database Per Service часто противопоставляют другой шаблон — **Shared Database («Разделяемая база данных»).** По сути, он представляет собой антипаттерн и подразумевает использование одного хранилища данных несколькими микросервисами. Его допускается использовать на начальных стадиях миграции на микросервисную архитектуру или в очень небольших приложениях, разрабатываемых одной командой (2–3 микросервиса).

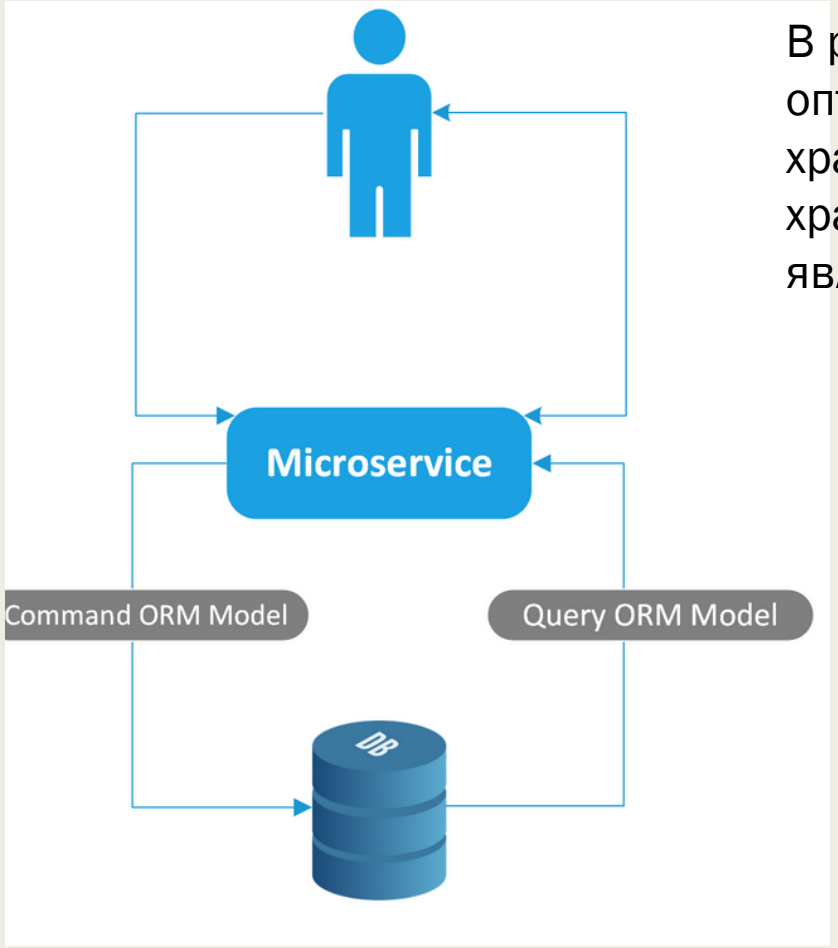
Шаблон «API-композиция» (API Composition)



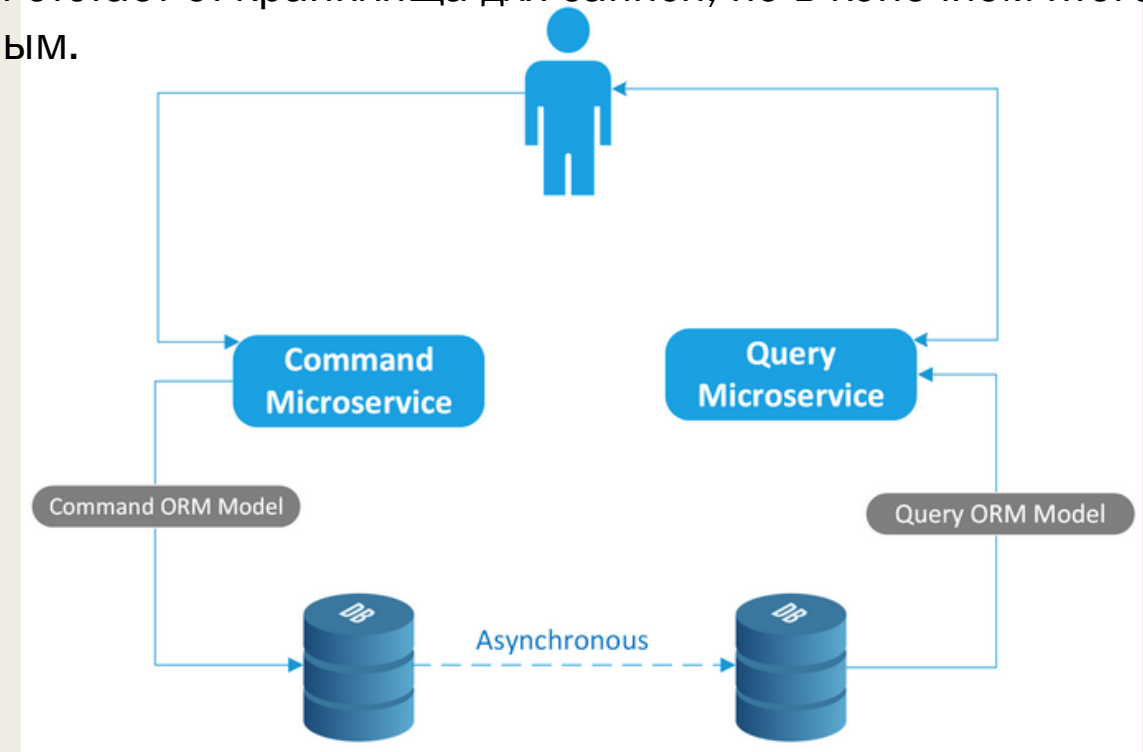
Он предлагает создать отдельное API, которое будет вызывать необходимые сервисы, владеющие данными, и выполнять соединение полученных от них результатов в памяти.

Шаблон «Разделение команд и запросов» (Command Query Responsibility Segregation, CQRS)

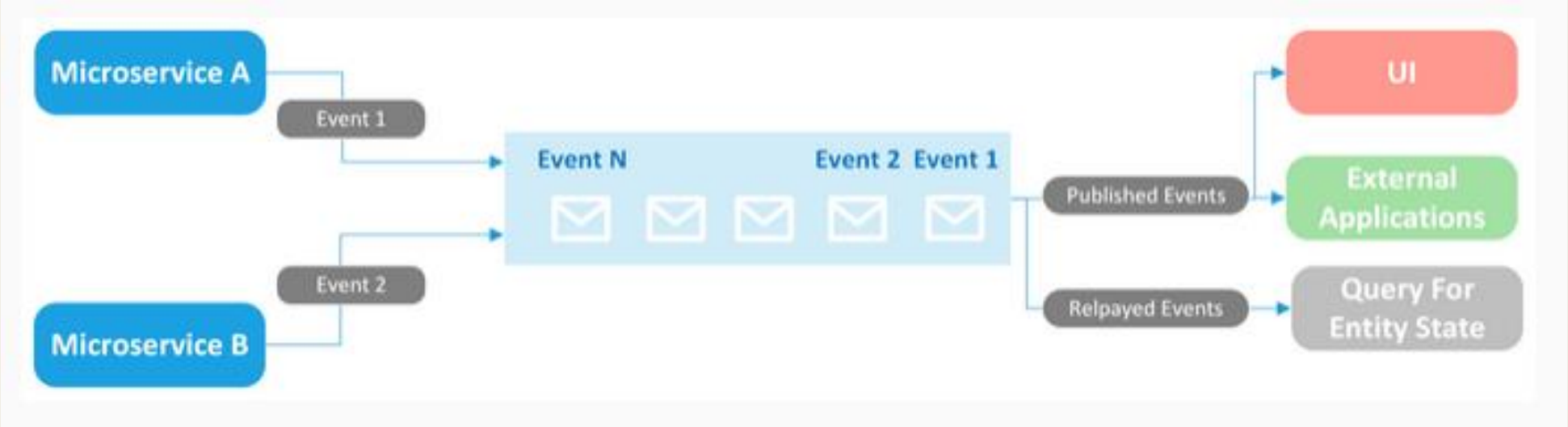
Этот паттерн предлагает отделить изменение данных (Command) от чтения данных (Query). Шаблон CQRS имеет две формы: простую и расширенную.
В простой форме для чтения и записи используются отдельные модели ORM (Object-Relational Mapping), но общее хранилище данных.



В расширенной форме используются разные хранилища данных, оптимизированные для записи и чтения данных. Данные копируются из хранилища для записи в хранилище для чтения асинхронно. В результате хранилище для чтения отстает от хранилища для записи, но в конечном итоге является согласованным.



Шаблон «Поиск событий» (Event Sourcing)

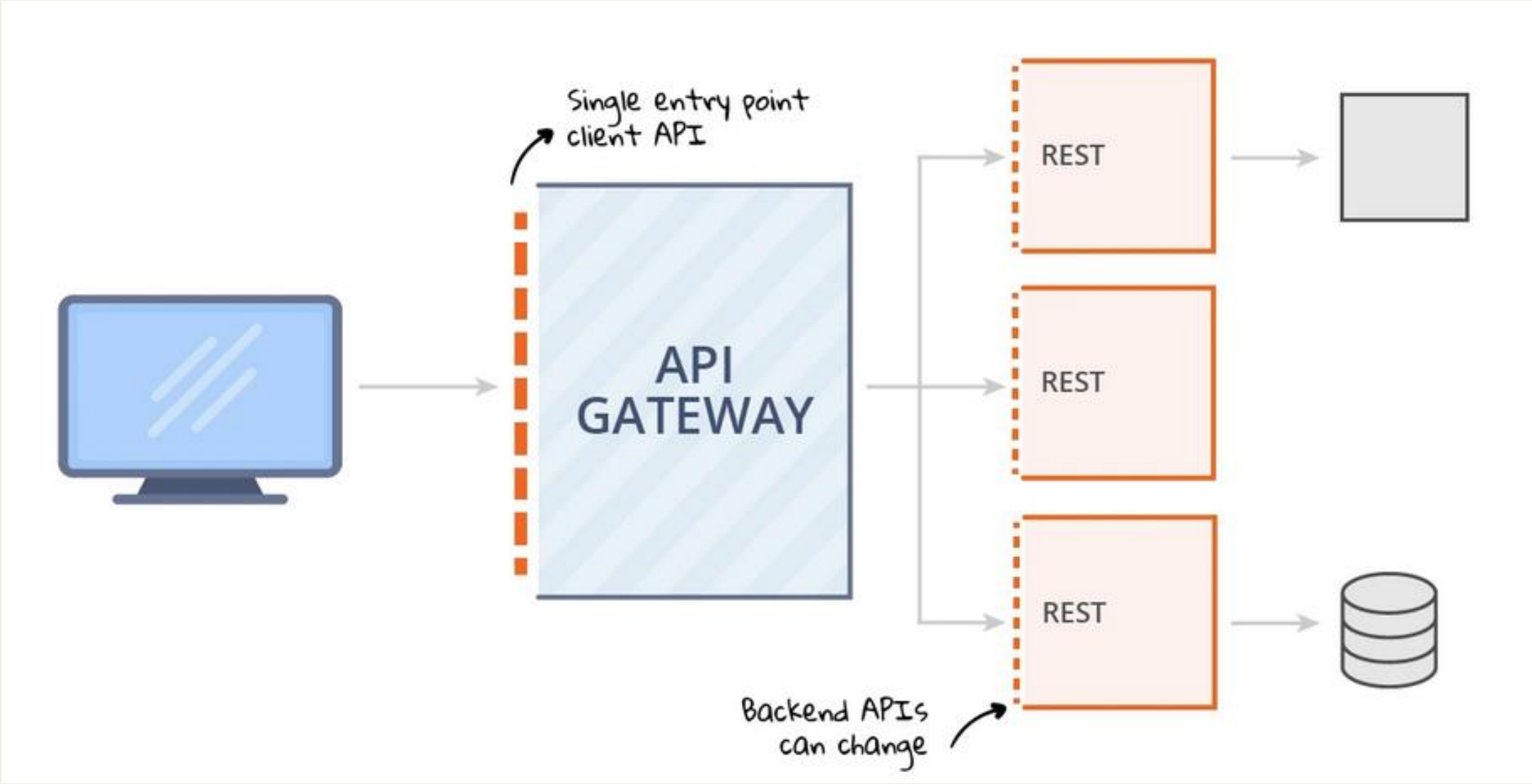


- В традиционных базах данных объект с текущим состоянием сохраняется напрямую. При использовании шаблона Event Sourcing вместо объектов сохраняются события, изменяющие их состояния. Итоговое состояние объекта можно получить путем повторной обработки серии событий, пришедших за определенное время. Различные службы могут воспроизводить события из хранилища событий, чтобы вычислить соответствующее состояние своих хранилищ данных. Для реализации хранилища событий обычно применяется шаблон CQRS.

Шаблон «Сага» (Saga)

- Этот паттерн предназначен для управления распределенными транзакциями в микросервисной архитектуре, где применение традиционного протокола двухфазной фиксации транзакций (Two-phase commit protocol, 2PC) становится трудноосуществимым.
- При использовании паттерна каждая локальная транзакция обновляет данные в хранилище в рамках одного микросервиса и публикует событие или сообщение, которые, в свою очередь, запускают следующую локальную транзакцию и так далее. Если локальная транзакция завершается с ошибкой, выполняется серия компенсирующих транзакций, которые отменяют изменения предыдущих транзакций.
- Для координации транзакций существует два основных способа:
  - ***Хореография.*** *Децентрализованная координация, при которой каждый микросервис прослушивает события/сообщения другого микросервиса и решает, следует предпринять действие или нет.*
  - ***Оркестровка.*** *Централизованная координация, при которой отдельный компонент (оркестратор) сообщает микросервисам, какое действие необходимо выполнить далее.*

Шаблоны API-шлюз

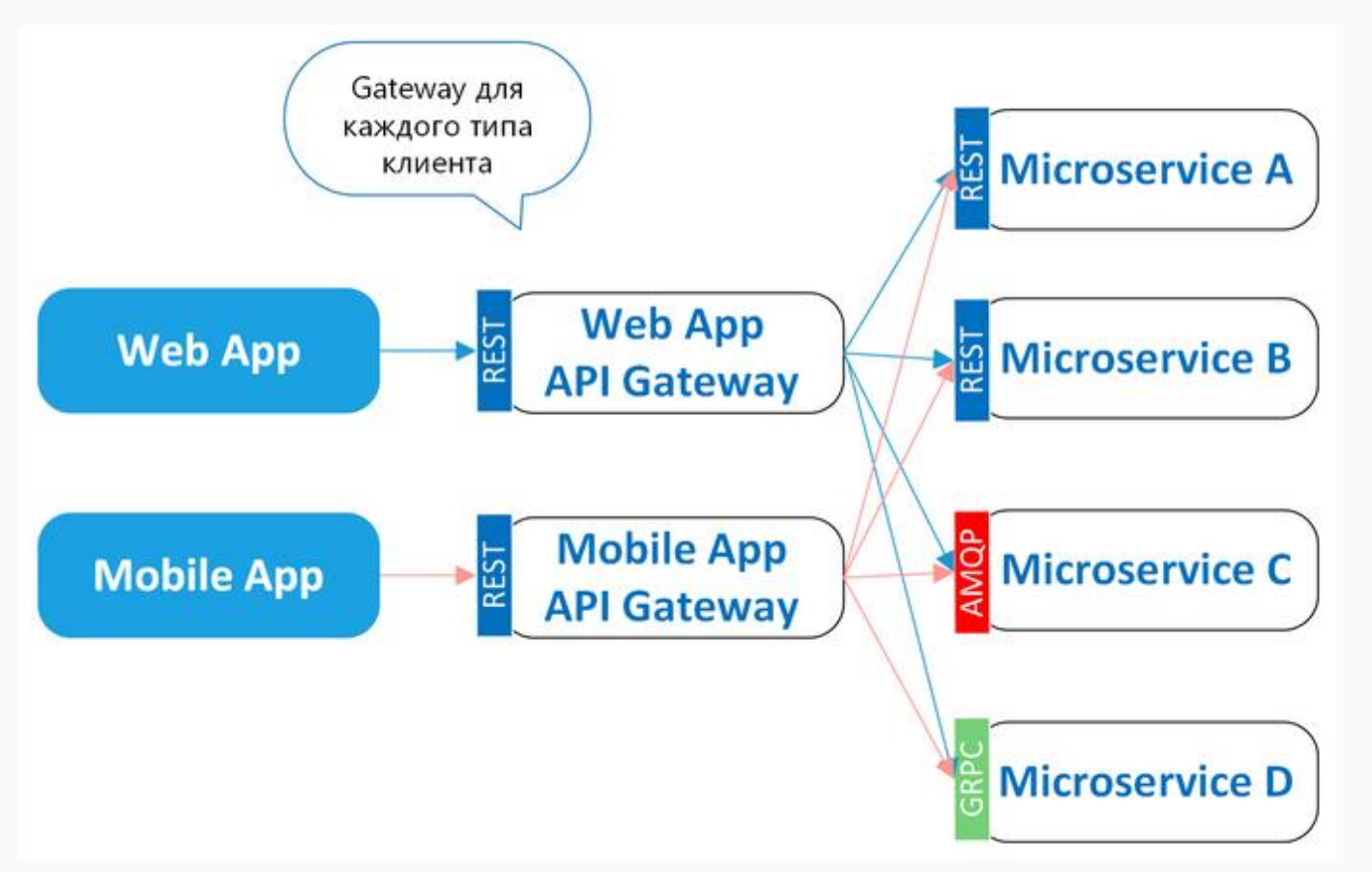


Шаблон API-шлюз

- В зависимости от конкретной цели использования паттерна иногда выделяют следующие его разновидности:
  - ***Gateway Routing.*** *Шлюз используется как обратный Proxy, перенаправляющий запросы клиента на соответствующий сервис.*
  - ***Gateway Aggregation.*** *Шлюз используется для разветвления клиентского запроса на несколько микросервисов и возвращения агрегированных ответов клиенту.*
  - ***Gateway Offloading.*** *Шлюз решает сквозные задачи, которые являются общими для сервисов: аутентификация, авторизация, SSL, ведение журналов и так далее.*
- Применение паттерна сокращает число вызовов, обеспечивает независимость клиента от протоколов, используемых в сервисах: REST, AMQP, gRPC и так далее, обеспечивает централизованное управление сквозной функциональностью. Однако шлюз может стать единой точкой отказа, требует тщательного мониторинга и при отсутствии масштабирования бывает узким местом системы.

https://mcs.mail.ru/blog/26-osnovnyh-patternov-mikroservisnoj-razrabotki

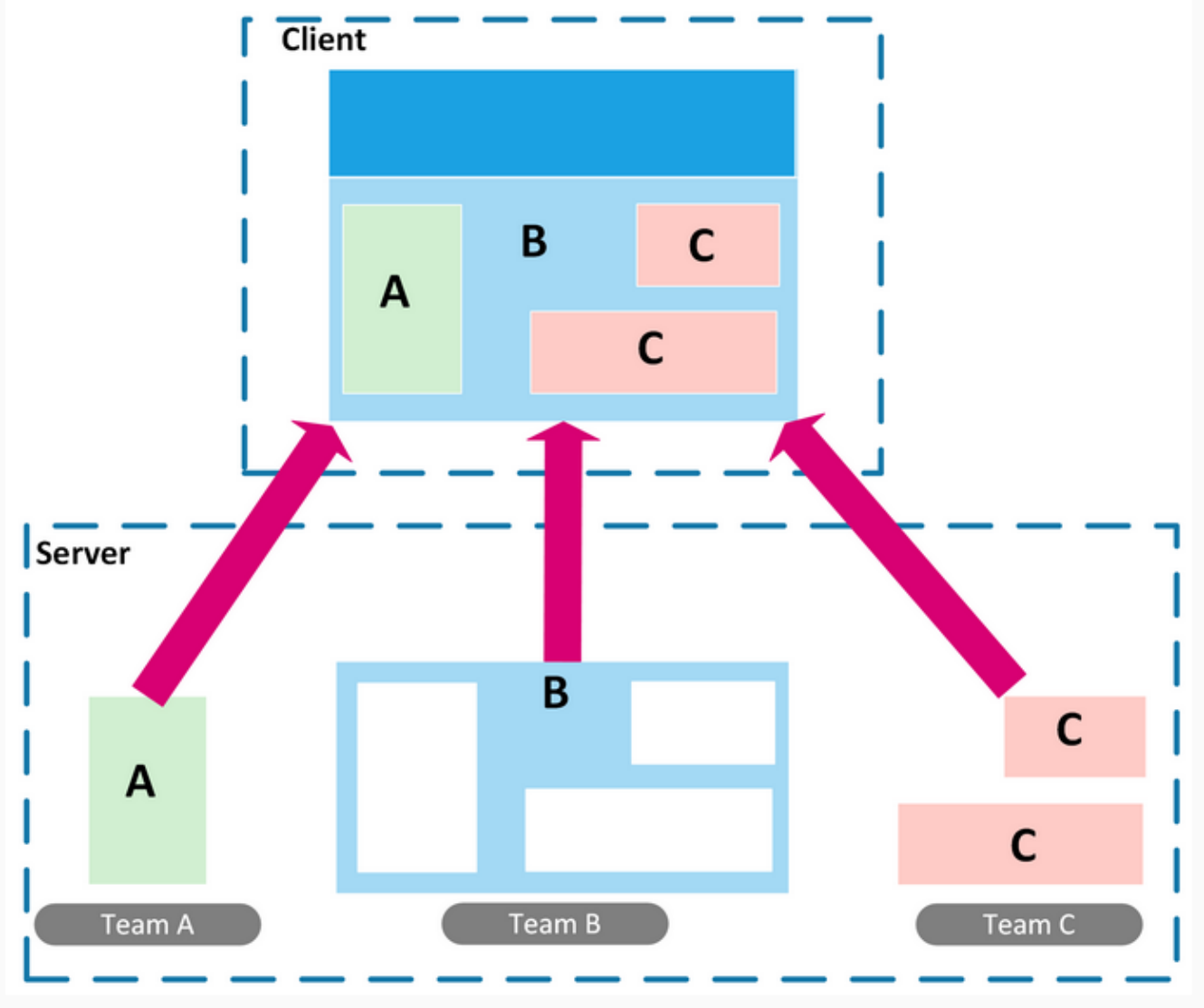
Шаблон «Бэкенды для фронтендов» (Backends for Frontends, BFF)



Этот паттерн является вариантом реализации шаблона API Gateway. Он также обеспечивает дополнительный уровень между микросервисами и клиентами, но вместо одной точки входа вводит несколько шлюзов для каждого типа клиента: Web, Mobile, Desktop и так далее.

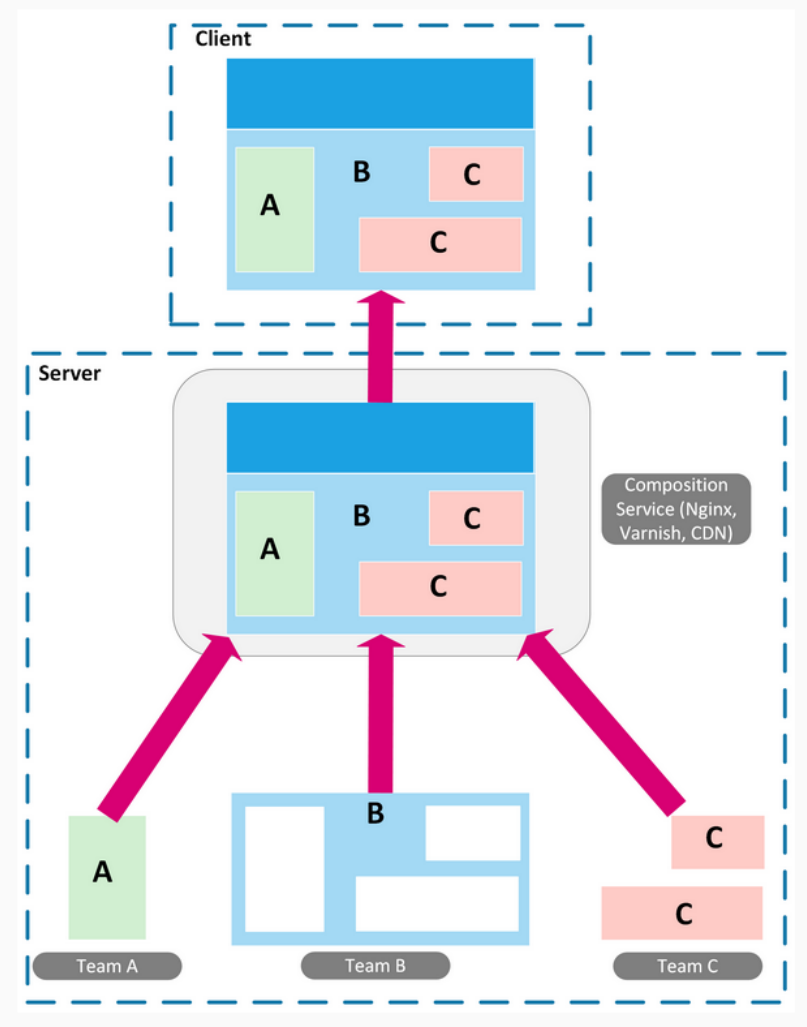
С помощью паттерна можно добавить API, адаптированные к потребностям каждого клиента, избавившись от хранения большого количества ненужных настроек в одном месте. Но шаблон не стоит применять в тех случаях, когда разница в требованиях к API у разных типов клиентов незначительна либо приложение само по себе небольшое: это приведет лишь к дублированию кода и увеличению числа компонентов.

Шаблон «Сборка пользовательского интерфейса на стороне клиента» (Client-Side UI Composition)



При использовании этого шаблона разметка HTML создается и обновляется непосредственно в браузере. Каждый экран/страница пользовательского интерфейса разбивается на фрагменты, данные для которых получают различные микросервисы. Каждый такой фрагмент, по сути, представляет собой мини-приложение, которое может отображать и обновлять свою разметку независимо от остальной части страницы.

Шаблон «Сборка фрагментов страниц на стороне сервера» (Server-Side Page Fragment Composition)



При использовании этого шаблона сборка фрагментов пользовательского интерфейса происходит на сервере, а клиентская часть получает уже полностью собранную страницу, благодаря чему достигается более высокая скорость загрузки. Сборка обычно выполняется отдельной службой, которая находится между браузером и серверами приложений: Nginx, Varnish, CDN.

Пример реализации API Gateway